

8-5 · 데이터 파이프라인 구축 및 최종 RAG 챗봇 개발 · 실습 1H

엔드투엔드 검증·시연

새 문서 하나가 답변에 나오는가 — 전 구간의 증명

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1시간 뒤에는

- ① 새 문서 투입 → DAG 실행 → 챗봇 답변 반영을 처음부터 끝까지 성공시킨다
- ② 두 번 돌려 points_count 가 늘지 않는 것(먹등)을 확인한다
- ③ Qdrant 를 내렸다 올려 retries 가 스스로 회복하는 것을 본다
- ④ 산출물을 코드·README·requirements·env.example 로 패키징한다

재기 전의 값을 남기는 것이 검증의 기본 — 기준선부터 적어 두세요

여덟 단계 — 한 줄 시나리오를 처음부터 끝까지

실무라면 투입구는 S3 버킷이나 업로드 API — 오늘은 incoming/ 폴더입니다

1·2. 기준선	3. 투입	4·5. 실행·확인	6·7. 멎등·회복	8. 패키징
points_count 「모른다」 대조군	incoming/ lakehouse.txt	trigger → sleep 90 다시 물어보기	두 번 돌리기 Qdrant stop	requirements .env.example

1~3단계

기준선, 대조군, 그리고 투입

1

기준선을 적는다

`collections/lecture_docs` 의 `points_count` 를 적어 둔다

2

챗봇에게 먼저 묻는다

「레이크하우스가 뭐야?」 — 지금은 모른다고 답해야 한다(대조군)

3

새 문서를 투입한다

`mkdir -p dags/data/incoming · cat > .../lakehouse.txt · ls` 로 확인

대조군이 있어야

답의 변화를
증명한다

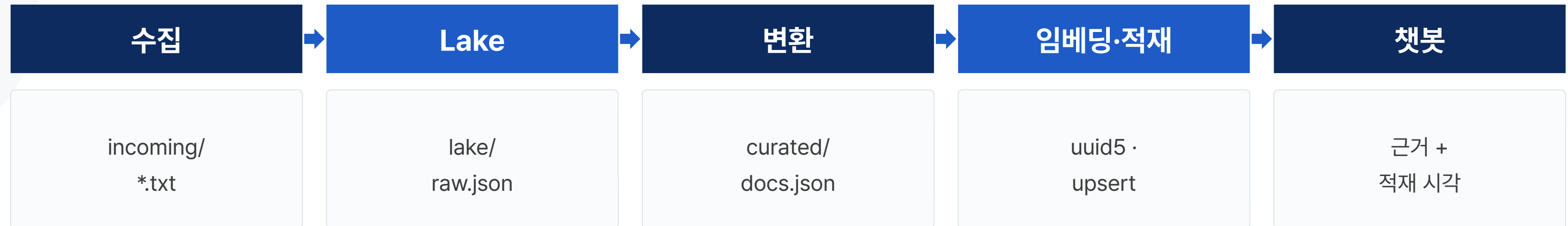
DAG 를 돌리고 다시 묻는다

CLI 별칭	CLI="docker compose exec airflow-scheduler airflow"
임포트 오류	\$CLI dags list-import-errors
활성화	\$CLI dags unpause rag_pipeline_final
실행	\$CLI dags trigger rag_pipeline_final
기다리기	sleep 90
세기	curl -s ../collections/lecture_docs grep -o '"points_count":[0-9]*'
다시 묻기	챗봇 새로고침 → 「레이크하우스가 뭐야?」

Grid 에서 세 태스크가 초록 · points_count 가 1단계보다 늘어 있다 · 이번엔 방금 넣은 문서를 근거로 답하고 적재 시각이 보인다

5단계

이 한 번의 문답이 전 구간이 살아 있다는 증명



근거 보기를 펼치면 방금 넣은 그 문장과 적재 시각이 보인다

어느 한 구간이라도 끊기면 실패한다 — 그래서 단계별 확인보다 강하다

두 번 돌려 세어 본다 — 멍등의 증거

최종본 — 내용 기반 uuid5

before → after 숫자가 그대로면 멍등

- before=\$(curl ... points_count)
- \$CLI dags trigger → sleep 90
- after=... · echo "\$before → \$after"

VS

8-3 버전 — 순번 id 였다면

같은 문서가 다른 id 로 들어가 늘어났을 것

- 실행마다 points_count 가 는다
- 같은 문서가 중복 검색된다
- 이론에서 본 「엉뚱한 자리 덮어쓰기」

Qdrant 를 내리고 돌려 재시도를 눈으로

retries=2, retry_delay 3분이 실제로 무엇을 해 주는지 봅니다

Qdrant 정지	트리거	Grid 관찰	60초 뒤 start	회복 확인
docker compose stop qdrant	dags trigger (최종 DAG)	embed_and_load 노랑(재시도 대기)	docker compose start qdrant	앞 둘은 초록 그대로 셋째만 재시도 성공

제출 폴더 다섯 파일과 마지막 grep

파일	어떻게
rag_pipeline_final.py	cp ~/airflow-lab/dags/... ./
app.py	오늘 고친 챗봇 그대로
requirements.txt	pip freeze > requirements.txt
.env.example	printf 로 키 이름만
README	아키텍처 그림·실행 순서·검증
.env(실제 키)	제출물·저장소에 넣지 않는다
마지막 확인	grep -rn "sk-" . → 비어 있어야

결과가 비어 있으면 제출 준비 완료 — 팀별 시연은 「투입 → 트리거 → 답변 반영」을 처음부터 끝까지 한 번에

이 1시간이 끝날 때 확인할 것

- 1 적어 둔 기준선보다 `points_count` 가 늘었다
- 2 「레이크하우스가 뭐야?」— 전에는 모른다, 후에는 근거와 적재 시각을 보인다
- 3 두 번째 트리거 뒤 `before` → `after` 숫자가 그대로다
- 4 Qdrant 를 올리자 `embed_and_load` 가 스스로 회복했다
- 5 제출 폴더 다섯 파일 · `grep "sk-"` 결과가 비어 있다

8-5 · 실습 1H 마무리

다음은 과정 회고·산출물 제출 확인 (0.5H)

8주 동안 만든 것을 돌아보고, 주차별 산출물이 제출 형태로 갖춰졌는지 확인합니다

✓ 무엇을 만들었나 — 플랫폼·파이프라인·챗봇

✓ 주차별 산출물 제출 확인 표

✓ 자주 겪는 오류 네 가지

✓ 다음에 공부할 것과 심화 과제

가져갈 것

시연 가능한 제출 폴더(키 없음)